

古筝智能体验系统

数据库设计说明（含全部字段注释）

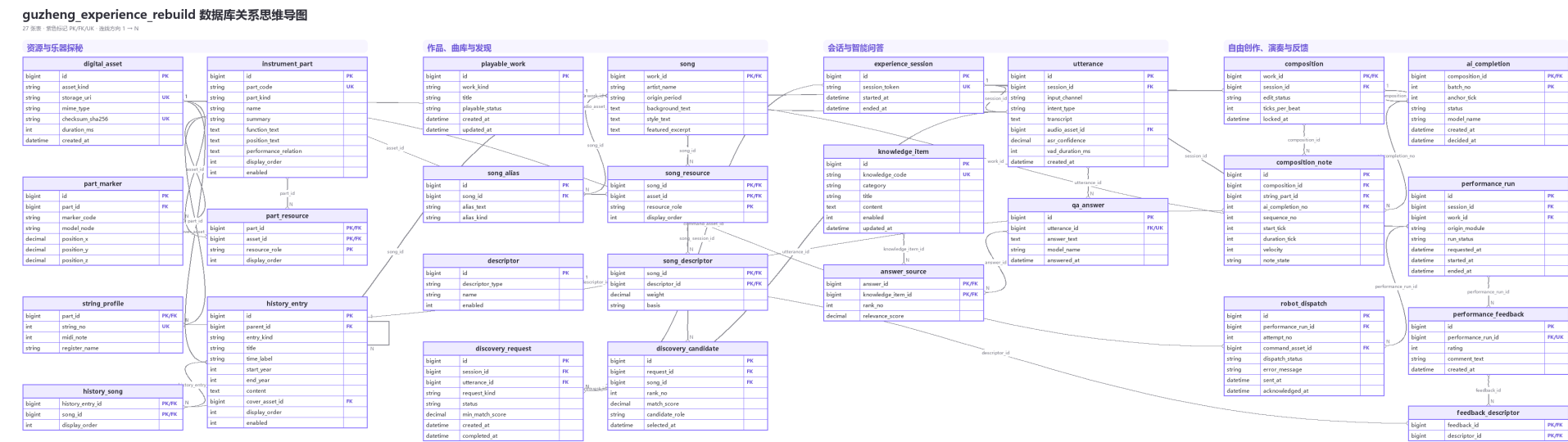
MySQL 8.0 · JDK 17 / Spring Boot 项目配套数据库

数据库名称: guzheng_experience_rebuild

文档内容: 关系图、表用途、字段字典、带注释建表代码

1. 数据库关系图

关系图中的 PK 表示主键，FK 表示外键，UK 表示唯一键；连线表示表之间的一对多或一对一关系。



2. 设计原则

- 1. 歌曲和自由创作统一抽象为 playable_work，机器人演奏只引用一个 work_id。
- 2. 图片、音频、模型、曲谱和控制指令统一保存在 digital_asset，业务表只保存资源外键。
- 3. 文字输入与语音识别结果统一保存在 utterance，避免问答和点歌各保存一份输入内容。
- 4. 人工音符和 AI 建议音符统一保存在 composition_note，通过状态区分是否采用。
- 5. 风格、情绪、场景和反馈词统一保存在 descriptor，避免重复保存标签文字。

3. 表结构与字段说明

下面按表逐一说明字段含义。代码块中的 COMMENT 内容与字段说明表保持一致，可直接用于 MySQL 建库。

3.1 digital_asset

统一保存图片、音频、3D 模型、曲谱和机器人指令等数字资源。

字段名	数据类型	定义提示	字段含义
id	BIGINT	PK / 必填 / 自增	数字资源主键
asset_kind	VARCHAR(24)	必填	资源大类，例如图片、音频、模型、曲谱或指令
storage_uri	VARCHAR(700)	UK / 必填	资源在本地或对象存储中的唯一访问地址
mime_type	VARCHAR(100)	可选/按定义	资源的 MIME 类型，例如 image/png 或 audio/mpeg
checksum_sha256	CHAR(64)	UK	资源文件的 SHA-256 校验值，用于判重和完整性校验
duration_ms	INT	可选/按定义	音频或视频资源时长，单位为毫秒
created_at	DATETIME(3)	必填	资源记录创建时间

建表代码

```
CREATE TABLE `digital_asset` (  
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '数字资源主键',  
  `asset_kind` VARCHAR(24) NOT NULL COMMENT '资源大类, 例如图片、音频、模型、曲谱或指令',  
  `storage_uri` VARCHAR(700) NOT NULL COMMENT '资源在本地或对象存储中的唯一访问地址',  
  `mime_type` VARCHAR(100) DEFAULT NULL COMMENT '资源的 MIME 类型, 例如 image/png 或 audio/mpeg',  
  `checksum_sha256` CHAR(64) DEFAULT NULL COMMENT '资源文件的 SHA-256 校验值, 用于判重和完整性校验',  
  `duration_ms` INT UNSIGNED DEFAULT NULL COMMENT '音频或视频资源时长, 单位为毫秒',  
  `created_at` DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) COMMENT '资源记录创建时间',  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `uk_asset_storage_uri` (`storage_uri`),  
  UNIQUE KEY `uk_asset_checksum` (`checksum_sha256`),  
  CONSTRAINT `ck_asset_kind` CHECK (`asset_kind` IN ('IMAGE','AUDIO','MODEL','SCORE','COMMAND','OTHER')),  
  CONSTRAINT `ck_asset_duration` CHECK (`duration_ms` IS NULL OR `duration_ms` >= 0)  
) ENGINE=InnoDB COMMENT='统一数字资源';
```

3.2 experience_session

记录用户从进入系统到离开系统的一次完整体验过程。

字段名	数据类型	定义提示	字段含义
id	BIGINT	PK / 必填 / 自增	体验会话主键
session_token	CHAR(36)	UK / 必填	前后端识别本次体验的唯一会话令牌
started_at	DATETIME(3)	必填	用户开始本次体验的时间
ended_at	DATETIME(3)	可选/按定义	用户结束本次体验的时间，未结束时为空

建表代码

```
CREATE TABLE `experience_session` (  
  `id` BIGINT UNSIGNED NOT NULL AUTO INCREMENT COMMENT '体验会话主键',  
  `session_token` CHAR(36) NOT NULL COMMENT '前后端识别本次体验的唯一会话令牌',  
  `started_at` DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) COMMENT '用户开始本次体验的时间',  
  `ended_at` DATETIME(3) DEFAULT NULL COMMENT '用户结束本次体验的时间，未结束时为空',  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `uk_session_token` (`session_token`),  
  KEY `idx_session_started_at` (`started_at`),  
  CONSTRAINT `ck_session_time` CHECK (`ended_at` IS NULL OR `ended_at` >= `started_at`)  
) ENGINE=InnoDB COMMENT='用户一次完整体验会话';
```

3.3 instrument_part

保存普通古筝部件和琴弦共同拥有的基础说明。

字段名	数据类型	定义提示	字段含义
id	BIGINT	PK / 必填 / 自增	古筝部件主键
part_code	VARCHAR(64)	UK / 必填	部件业务编码，例如 BRIDGE 或 STRING_01
part_kind	VARCHAR(16)	必填	部件类型，普通部件 COMPONENT 或琴弦 STRING
name	VARCHAR(100)	必填	页面显示的部件名称
summary	VARCHAR(500)	必填	热点卡片中显示的部件简要介绍
function_text	TEXT	可选/按定义	部件作用的详细说明
position_text	TEXT	可选/按定义	部件在古筝上的位置说明
performance_relation	TEXT	可选/按定义	该部件与演奏方式或音色之间的关系
display_order	SMALLINT	必填	部件在页面列表中的显示顺序，数字越小越靠前
enabled	BOOLEAN	必填	部件是否启用，1 为启用，0 为停用

建表代码

```
CREATE TABLE `instrument_part` (  
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '古筝部件主键',  
  `part_code` VARCHAR(64) NOT NULL COMMENT '部件业务编码，例如 BRIDGE 或 STRING_01',  
  `part_kind` VARCHAR(16) NOT NULL COMMENT '部件类型，普通部件 COMPONENT 或琴弦 STRING',  
  `name` VARCHAR(100) NOT NULL COMMENT '页面显示的部件名称',  
  `summary` VARCHAR(500) NOT NULL COMMENT '热点卡片中显示的部件简要介绍',  
  `function_text` TEXT COMMENT '部件作用的详细说明',  
  `position_text` TEXT COMMENT '部件在古筝上的位置说明',  
  `performance_relation` TEXT COMMENT '该部件与演奏方式或音色之间的关系',  
  `display_order` SMALLINT UNSIGNED NOT NULL DEFAULT 0 COMMENT '部件在页面列表中的显示顺序，数字越小越靠前',  
  `enabled` BOOLEAN NOT NULL DEFAULT TRUE COMMENT '部件是否启用，1 为启用，0 为停用',  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `uk_part_code` (`part_code`),  
  KEY `idx_part_kind_order` (`part_kind`, `display_order`),  
  CONSTRAINT `ck_part_kind` CHECK (`part_kind` IN ('COMPONENT','STRING'))  
) ENGINE=InnoDB COMMENT='古筝部件与琴弦公共信息';
```

3.4 part_marker

保存部件在 3D 古筝模型中的热点位置。

字段名	数据类型	定义提示	字段含义
id	BIGINT	PK / 必填 / 自增	部件热点主键
part_id	BIGINT	FK / 联合UK / 必填	热点对应的古筝部件编号
marker_code	VARCHAR(64)	联合UK / 必填	同一部件下热点的业务编码
model_node	VARCHAR(128)	可选/按定义	3D 模型中对应的节点名称
position_x	DECIMAL(10,6)	可选/按定义	热点在 3D 坐标系中的 X 坐标
position_y	DECIMAL(10,6)	可选/按定义	热点在 3D 坐标系中的 Y 坐标
position_z	DECIMAL(10,6)	可选/按定义	热点在 3D 坐标系中的 Z 坐标

建表代码

```
CREATE TABLE `part_marker` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '部件热点主键',
  `part_id` BIGINT UNSIGNED NOT NULL COMMENT '热点对应的古筝部件编号',
  `marker_code` VARCHAR(64) NOT NULL COMMENT '同一部件下热点的业务编码',
  `model_node` VARCHAR(128) DEFAULT NULL COMMENT '3D 模型中对应的节点名称',
  `position_x` DECIMAL(10,6) DEFAULT NULL COMMENT '热点在 3D 坐标系中的 X 坐标',
  `position_y` DECIMAL(10,6) DEFAULT NULL COMMENT '热点在 3D 坐标系中的 Y 坐标',
  `position_z` DECIMAL(10,6) DEFAULT NULL COMMENT '热点在 3D 坐标系中的 Z 坐标',
  PRIMARY KEY (`id`),
  UNIQUE KEY `uk_part_marker_code` (`part_id`, `marker_code`),
  CONSTRAINT `fk_marker_part` FOREIGN KEY (`part_id`)
  REFERENCES `instrument_part` (`id`) ON DELETE CASCADE
) ENGINE=InnoDB COMMENT='3D 模型中的部件热点';
```

3.5 part_resource

建立古筝部件与图片、试听音频或模型资源之间的关系。

字段名	数据类型	定义提示	字段含义
part_id	BIGINT	联合PK / FK / 必填	使用资源的古筝部件编号
asset_id	BIGINT	联合PK / FK / 必填	被使用的数字资源编号
resource_role	VARCHAR(24)	联合PK / 必填	资源在该部件中的用途，例如详情图、试听音频或模型
display_order	SMALLINT	必填	同类资源的显示或播放顺序

建表代码

```
CREATE TABLE `part_resource` (  
  `part_id` BIGINT UNSIGNED NOT NULL COMMENT '使用资源的古筝部件编号',  
  `asset_id` BIGINT UNSIGNED NOT NULL COMMENT '被使用的数字资源编号',  
  `resource_role` VARCHAR(24) NOT NULL COMMENT '资源在该部件中的用途，例如详情图、试听音频或模型',  
  `display_order` SMALLINT UNSIGNED NOT NULL DEFAULT 0 COMMENT '同类资源的显示或播放顺序',  
  PRIMARY KEY (`part_id`, `asset_id`, `resource_role`),  
  KEY `idx_part_resource_asset` (`asset_id`),  
  CONSTRAINT `fk_part_resource_part` FOREIGN KEY (`part_id`)  
    REFERENCES `instrument_part` (`id`) ON DELETE CASCADE,  
  CONSTRAINT `fk_part_resource_asset` FOREIGN KEY (`asset_id`)  
    REFERENCES `digital_asset` (`id`) ON DELETE RESTRICT,  
  CONSTRAINT `ck_part_resource_role` CHECK (`resource_role` IN ('DETAIL_IMAGE','DEMO_AUDIO','MODEL'))  
) ENGINE=InnoDB COMMENT='部件关联资源';
```


3.6 string_profile

保存琴弦独有的弦号、MIDI 音高和音区信息。

字段名	数据类型	定义提示	字段含义
part_id	BIGINT	PK / FK / 必填	琴弦对应的部件编号，同时也是本表主键
string_no	TINYINT	UK / 必填	琴弦编号，例如第 1 弦、第 2 弦
midi_note	TINYINT	必填	琴弦标准音高对应的 MIDI 音符编号
register_name	VARCHAR(30)	必填	琴弦所属音区名称，例如低音区或中音区

建表代码

```
CREATE TABLE `string_profile` (  
  `part_id` BIGINT UNSIGNED NOT NULL COMMENT '琴弦对应的部件编号，同时也是本表主键';  
  `string_no` TINYINT UNSIGNED NOT NULL COMMENT '琴弦编号，例如第 1 弦、第 2 弦';  
  `midi_note` TINYINT UNSIGNED NOT NULL COMMENT '琴弦标准音高对应的 MIDI 音符编号';  
  `register_name` VARCHAR(30) NOT NULL COMMENT '琴弦所属音区名称，例如低音区或中音区';  
  PRIMARY KEY (`part_id`),  
  UNIQUE KEY `uk_string_no` (`string_no`),  
  CONSTRAINT `fk_string_part` FOREIGN KEY (`part_id`)  
    REFERENCES `instrument_part` (`id`) ON DELETE CASCADE,  
  CONSTRAINT `ck_string_no` CHECK (`string_no` BETWEEN 1 AND 64),  
  CONSTRAINT `ck_string_midi` CHECK (`midi_note` BETWEEN 0 AND 127)  
) ENGINE=InnoDB COMMENT='琴弦专有音高信息';
```

3.7 history_entry

用层级结构保存历史时期、历史事件和文化故事。

字段名	数据类型	定义提示	字段含义
id	BIGINT	PK / 必填 / 自增	历史内容主键
parent_id	BIGINT	FK	上级历史内容编号，用于组成时期、事件和故事的层级
entry_kind	VARCHAR(16)	必填	内容类型，历史时期 PERIOD、事件 EVENT 或故事 STORY
title	VARCHAR(200)	必填	历史时期、事件或故事的标题
time_label	VARCHAR(100)	可选/按定义	页面显示的年代文字，例如先秦时期或现代
start_year	SMALLINT	可选/按定义	历史内容的起始年份，无法确定时空
end_year	SMALLINT	可选/按定义	历史内容的结束年份，无法确定时空
content	TEXT	必填	历史内容的完整介绍
cover_asset_id	BIGINT	FK	历史内容使用的封面或配图资源编号
display_order	SMALLINT	必填	同一级历史内容在时间轴中的显示顺序
enabled	BOOLEAN	必填	历史内容是否在前端展示

建表代码

```
CREATE TABLE `history_entry` (
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '历史内容主键',
  `parent_id` BIGINT UNSIGNED DEFAULT NULL COMMENT '上级历史内容编号，用于组成时期、事件和故事的层级',
  `entry_kind` VARCHAR(16) NOT NULL COMMENT '内容类型，历史时期 PERIOD、事件 EVENT 或故事 STORY',
  `title` VARCHAR(200) NOT NULL COMMENT '历史时期、事件或故事的标题',
  `time_label` VARCHAR(100) DEFAULT NULL COMMENT '页面显示的年代文字，例如先秦时期或现代',
  `start_year` SMALLINT DEFAULT NULL COMMENT '历史内容的起始年份，无法确定时空',
  `end_year` SMALLINT DEFAULT NULL COMMENT '历史内容的结束年份，无法确定时空',
  `content` TEXT NOT NULL COMMENT '历史内容的完整介绍',
  `cover_asset_id` BIGINT UNSIGNED DEFAULT NULL COMMENT '历史内容使用的封面或配图资源编号',
  `display_order` SMALLINT UNSIGNED NOT NULL DEFAULT 0 COMMENT '同一级历史内容在时间轴中的显示顺序',
  `enabled` BOOLEAN NOT NULL DEFAULT TRUE COMMENT '历史内容是否在前端展示',
  PRIMARY KEY (`id`),
  KEY `idx_history_parent_order` (`parent_id`, `display_order`),
  KEY `idx_history_kind` (`entry_kind`),
  KEY `idx_history_cover_asset` (`cover_asset_id`),
  CONSTRAINT `fk_history_parent` FOREIGN KEY (`parent_id`)
  REFERENCES `history_entry` (`id`) ON DELETE CASCADE,
  CONSTRAINT `fk_history_cover` FOREIGN KEY (`cover_asset_id`)
  REFERENCES `digital_asset` (`id`) ON DELETE SET NULL,
  CONSTRAINT `ck_history_kind` CHECK (`entry_kind` IN ('PERIOD','EVENT','STORY')),
  CONSTRAINT `ck_history_years` CHECK (`start_year` IS NULL OR `end_year` IS NULL OR `end_year` >= `start_year`)
) ENGINE=InnoDB COMMENT='历史时间轴、事件与故事';
```

3.8 playable_work

统一表示能够交给古筝机器人演奏的歌曲或自由创作作品。

字段名	数据类型	定义提示	字段含义
id	BIGINT	PK / 必填 / 自增	可演奏作品主键
work_kind	VARCHAR(16)	必填	作品类型，曲库歌曲 SONG 或自由创作 COMPOSITION
title	VARCHAR(200)	必填	歌曲或自由创作作品的统一标题
playable_status	VARCHAR(16)	必填	作品是否可演奏的状态：草稿、就绪或已归档
created_at	DATETIME(3)	必填	作品创建时间
updated_at	DATETIME(3)	必填	作品最后更新时间

建表代码

```
CREATE TABLE `playable_work` (  
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '可演奏作品主键',  
  `work_kind` VARCHAR(16) NOT NULL COMMENT '作品类型，曲库歌曲 SONG 或自由创作 COMPOSITION',  
  `title` VARCHAR(200) NOT NULL COMMENT '歌曲或自由创作作品的统一标题',  
  `playable_status` VARCHAR(16) NOT NULL DEFAULT 'DRAFT' COMMENT '作品是否可演奏的状态：草稿、就绪或已归档',  
  `created_at` DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) COMMENT '作品创建时间',  
  `updated_at` DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3) COMMENT '作品最后更新时间',  
  PRIMARY KEY (`id`),  
  KEY `idx_work_kind_status` (`work_kind`, `playable_status`),  
  KEY `idx_work_title` (`title`),  
  CONSTRAINT `ck_work_kind` CHECK (`work_kind` IN ('SONG','COMPOSITION')),  
  CONSTRAINT `ck_work_status` CHECK (`playable_status` IN ('DRAFT','READY','ARCHIVED'))  
) ENGINE=InnoDB COMMENT='所有可演奏作品的公共信息';
```

3.9 song

保存曲库歌曲独有的作者、背景和风格介绍。

字段名	数据类型	定义提示	字段含义
work_id	BIGINT	PK / FK / 必填	歌曲对应的可演奏作品编号，同时也是本表主键
artist_name	VARCHAR(160)	可选/按定义	曲目作者、作曲者或主要表演者名称
origin_period	VARCHAR(100)	可选/按定义	曲目产生或流行的历史时期
background_text	TEXT	可选/按定义	曲目的创作背景和文化故事
style_text	TEXT	可选/按定义	曲目风格特点的文字介绍，不作为筛选标签
featured_excerpt	TEXT	可选/按定义	适合在探秘页面展示的代表片段说明

建表代码

```
CREATE TABLE `song` (  
  `work_id` BIGINT UNSIGNED NOT NULL COMMENT '歌曲对应的可演奏作品编号，同时也是本表主键',  
  `artist_name` VARCHAR(160) DEFAULT NULL COMMENT '曲目作者、作曲者或主要表演者名称',  
  `origin_period` VARCHAR(100) DEFAULT NULL COMMENT '曲目产生或流行的历史时期',  
  `background_text` TEXT COMMENT '曲目的创作背景和文化故事',  
  `style_text` TEXT COMMENT '曲目风格特点的文字介绍，不作为筛选标签',  
  `featured_excerpt` TEXT COMMENT '适合在探秘页面展示的代表片段说明',  
  PRIMARY KEY (`work_id`),  
  CONSTRAINT `fk_song_work` FOREIGN KEY (`work_id`)  
    REFERENCES `playable_work` (`id`) ON DELETE CASCADE  
) ENGINE=InnoDB COMMENT='曲库歌曲专有信息';
```

3.10 song_alias

保存歌曲别名、作者别名以及语音模糊匹配词。

字段名	数据类型	定义提示	字段含义
id	BIGINT	PK / 必填 / 自增	歌曲别名主键
song_id	BIGINT	FK / 联合UK / 必填	别名所属的歌曲编号
alias_text	VARCHAR(200)	联合UK / 必填	用于搜索或语音匹配的别名文字
alias_kind	VARCHAR(16)	联合UK / 必填	别名类型：曲名别名、作者别名或其他别名

建表代码

```
CREATE TABLE `song_alias` (  
  `id` BIGINT UNSIGNED NOT NULL AUTO INCREMENT COMMENT '歌曲别名主键',  
  `song_id` BIGINT UNSIGNED NOT NULL COMMENT '别名所属的歌曲编号',  
  `alias_text` VARCHAR(200) NOT NULL COMMENT '用于搜索或语音匹配的别名文字',  
  `alias_kind` VARCHAR(16) NOT NULL DEFAULT 'OTHER' COMMENT '别名类型：曲名别名、作者别名或其他别名',  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `uk_song_alias` (`song_id`, `alias_text`, `alias_kind`),  
  KEY `idx_song_alias_text` (`alias_text`),  
  CONSTRAINT `fk_alias_song` FOREIGN KEY (`song_id`)  
    REFERENCES `song` (`work_id`) ON DELETE CASCADE,  
  CONSTRAINT `ck_alias_kind` CHECK (`alias_kind` IN ('TITLE','ARTIST','OTHER'))  
) ENGINE=InnoDB COMMENT='歌曲别名与模糊匹配词';
```

3.11 song_resource

建立歌曲与封面、试听、片段和曲谱资源之间的关系。

字段名	数据类型	定义提示	字段含义
song_id	BIGINT	联合PK / FK / 必填	使用资源的歌曲编号
asset_id	BIGINT	联合PK / FK / 必填	歌曲关联的数字资源编号
resource_role	VARCHAR(24)	联合PK / 必填	资源用途，例如封面、试听、展示片段或机器人曲谱
display_order	SMALLINT	必填	同类歌曲资源的展示顺序

建表代码

```
CREATE TABLE `song_resource` (  
  `song_id` BIGINT UNSIGNED NOT NULL COMMENT '使用资源的歌曲编号',  
  `asset_id` BIGINT UNSIGNED NOT NULL COMMENT '歌曲关联的数字资源编号',  
  `resource_role` VARCHAR(24) NOT NULL COMMENT '资源用途，例如封面、试听、展示片段或机器人曲谱',  
  `display_order` SMALLINT UNSIGNED NOT NULL DEFAULT 0 COMMENT '同类歌曲资源的展示顺序',  
  PRIMARY KEY (`song_id`, `asset_id`, `resource_role`),  
  KEY `idx_song_resource_asset` (`asset_id`),  
  CONSTRAINT `fk_song_resource_song` FOREIGN KEY (`song_id`)  
    REFERENCES `song` (`work_id`) ON DELETE CASCADE,  
  CONSTRAINT `fk_song_resource_asset` FOREIGN KEY (`asset_id`)  
    REFERENCES `digital_asset` (`id`) ON DELETE RESTRICT,  
  CONSTRAINT `ck_song_resource_role` CHECK (`resource_role` IN ('COVER','PREVIEW','EXCERPT','SCORE'))  
) ENGINE=InnoDB COMMENT='歌曲封面、试听、片段和曲谱资源';
```

3.12 history_song

关联历史内容与对应的代表曲目。

字段名	数据类型	定义提示	字段含义
history_entry_id	BIGINT	联合PK / FK / 必填	关联的历史内容编号
song_id	BIGINT	联合PK / FK / 必填	历史内容中出现的代表歌曲编号
display_order	SMALLINT	必填	代表曲目在当前历史内容中的显示顺序

建表代码

```
CREATE TABLE `history_song` (  
  `history_entry_id` BIGINT UNSIGNED NOT NULL COMMENT '关联的历史内容编号',  
  `song_id` BIGINT UNSIGNED NOT NULL COMMENT '历史内容中出现的代表歌曲编号',  
  `display_order` SMALLINT UNSIGNED NOT NULL DEFAULT 0 COMMENT '代表曲目在当前历史内容中的显示顺序',  
  PRIMARY KEY (`history_entry_id`, `song_id`),  
  KEY `idx_history_song_song` (`song_id`),  
  CONSTRAINT `fk_history_song_entry` FOREIGN KEY (`history_entry_id`)  
    REFERENCES `history_entry` (`id`) ON DELETE CASCADE,  
  CONSTRAINT `fk_history_song_song` FOREIGN KEY (`song_id`)  
    REFERENCES `song` (`work_id`) ON DELETE RESTRICT  
) ENGINE=InnoDB COMMENT='历史内容中的代表曲目';
```


3.13 descriptor

统一保存风格、情绪、场景和演奏反馈词。

字段名	数据类型	定义提示	字段含义
id	BIGINT	PK / 必填 / 自增	描述词主键
descriptor_type	VARCHAR(16)	联合UK / 必填	描述词类型：风格、情绪、场景或反馈
name	VARCHAR(60)	联合UK / 必填	描述词名称，例如古风、舒缓或欢快
enabled	BOOLEAN	必填	描述词是否可用于推荐和反馈

建表代码

```
CREATE TABLE `descriptor` (  
  `id` BIGINT UNSIGNED NOT NULL AUTO INCREMENT COMMENT '描述词主键',  
  `descriptor_type` VARCHAR(16) NOT NULL COMMENT '描述词类型：风格、情绪、场景或反馈',  
  `name` VARCHAR(60) NOT NULL COMMENT '描述词名称，例如古风、舒缓或欢快',  
  `enabled` BOOLEAN NOT NULL DEFAULT TRUE COMMENT '描述词是否可用于推荐和反馈',  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `uk_descriptor_type_name` (`descriptor_type`, `name`),  
  CONSTRAINT `ck_descriptor_type` CHECK (`descriptor_type` IN ('STYLE','MOOD','SCENE','FEEDBACK'))  
) ENGINE=InnoDB COMMENT='推荐与反馈共用描述词';
```

3.14 song_descriptor

保存歌曲具有哪些描述词以及描述词的匹配权重。

字段名	数据类型	定义提示	字段含义
song_id	BIGINT	联合PK / FK / 必填	被标记的歌曲编号
descriptor_id	BIGINT	联合PK / FK / 必填	歌曲具有的描述词编号
weight	DECIMAL(5,4)	必填	描述词与歌曲的匹配权重，取值范围为 0 到 1
basis	VARCHAR(16)	必填	标签建立依据，人工整理或冷启动预设

建表代码

```
CREATE TABLE `song_descriptor` (  
  `song_id` BIGINT UNSIGNED NOT NULL COMMENT '被标记的歌曲编号',  
  `descriptor_id` BIGINT UNSIGNED NOT NULL COMMENT '歌曲具有的描述词编号',  
  `weight` DECIMAL(5,4) NOT NULL DEFAULT 1.0000 COMMENT '描述词与歌曲的匹配权重，取值范围为 0 到 1',  
  `basis` VARCHAR(16) NOT NULL DEFAULT 'CURATED' COMMENT '标签建立依据，人工整理或冷启动预设',  
  PRIMARY KEY (`song_id`, `descriptor_id`),  
  KEY `idx_song_descriptor_descriptor` (`descriptor_id`, `weight`),  
  CONSTRAINT `fk_song_descriptor_song` FOREIGN KEY (`song_id`) REFERENCES `song` (`work_id`) ON DELETE CASCADE,  
  CONSTRAINT `fk_song_descriptor_word` FOREIGN KEY (`descriptor_id`) REFERENCES `descriptor` (`id`) ON DELETE RESTRICT,  
  CONSTRAINT `ck_song_descriptor_weight` CHECK (`weight` BETWEEN 0.0000 AND 1.0000),  
  CONSTRAINT `ck_song_descriptor_basis` CHECK (`basis` IN ('CURATED','COLD_START'))  
) ENGINE=InnoDB COMMENT='歌曲的推荐描述词';
```

3.15 utterance

统一保存用户的文字输入和语音识别结果。

字段名	数据类型	定义提示	字段含义
id	BIGINT	PK / 必填 / 自增	用户输入主键
session_id	BIGINT	FK	该输入所属的体验会话编号
input_channel	VARCHAR(12)	必填	输入渠道，文字 TEXT 或语音 VOICE
intent_type	VARCHAR(24)	必填	识别出的用户意图，例如问答、搜歌或推荐
transcript	TEXT	必填	用户输入的文字，语音输入时保存识别后的文本
audio_asset_id	BIGINT	FK	原始语音录音对应的数字资源编号
asr_confidence	DECIMAL(5,4)	可选/按定义	语音识别置信度，取值范围为 0 到 1
vad_duration_ms	INT	可选/按定义	端点检测得到的有效语音时长，单位为毫秒
created_at	DATETIME(3)	必填	用户输入产生的时间

建表代码

```
CREATE TABLE `utterance` (  
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '用户输入主键',  
  `session_id` BIGINT UNSIGNED DEFAULT NULL COMMENT '该输入所属的体验会话编号',  
  `input_channel` VARCHAR(12) NOT NULL COMMENT '输入渠道, 文字 TEXT 或语音 VOICE',  
  `intent_type` VARCHAR(24) NOT NULL COMMENT '识别出的用户意图, 例如问答、搜歌或推荐',  
  `transcript` TEXT NOT NULL COMMENT '用户输入的文字, 语音输入时保存识别后的文本',  
  `audio_asset_id` BIGINT UNSIGNED DEFAULT NULL COMMENT '原始语音录音对应的数字资源编号',  
  `asr_confidence` DECIMAL(5,4) DEFAULT NULL COMMENT '语音识别置信度, 取值范围为 0 到 1',  
  `vad_duration_ms` INT UNSIGNED DEFAULT NULL COMMENT '端点检测得到的有效语音时长, 单位为毫秒',  
  `created_at` DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) COMMENT '用户输入产生的时间',  
  PRIMARY KEY (`id`),  
  KEY `idx_utterance_session_time` (`session_id`, `created_at`),  
  KEY `idx_utterance_audio` (`audio_asset_id`),  
  CONSTRAINT `fk_utterance_session` FOREIGN KEY (`session_id`)  
    REFERENCES `experience_session` (`id`) ON DELETE SET NULL,  
  CONSTRAINT `fk_utterance_audio` FOREIGN KEY (`audio_asset_id`)  
    REFERENCES `digital_asset` (`id`) ON DELETE RESTRICT,  
  CONSTRAINT `ck_utterance_channel` CHECK (`input_channel` IN ('TEXT','VOICE')),  
  CONSTRAINT `ck_utterance_intent` CHECK (`intent_type` IN ('QA','SONG_SEARCH','RECOMMENDATION','OTHER')),  
  CONSTRAINT `ck_utterance_asr` CHECK (`asr_confidence` IS NULL OR `asr_confidence` BETWEEN 0.0000 AND 1.0000),  
  CONSTRAINT `ck_utterance_voice_fields` CHECK (  
    (`input_channel` = 'VOICE' AND `audio_asset_id` IS NOT NULL)  
    OR (`input_channel` = 'TEXT' AND `audio_asset_id` IS NULL AND `asr_confidence` IS NULL AND `vad_duration_ms` IS NULL)  
  )  
  ) ENGINE=InnoDB COMMENT='用户文本或语音输入';
```

3.16 knowledge_item

保存智能问答使用的古筝历史、乐理等知识条目。

字段名	数据类型	定义提示	字段含义
id	BIGINT	PK / 必填 / 自增	知识条目主键
knowledge_code	VARCHAR(64)	UK / 必填	知识条目的唯一业务编码
category	VARCHAR(60)	必填	知识分类，例如历史、结构、演奏技法或乐理
title	VARCHAR(200)	必填	知识条目标题
content	MEDIUMTEXT	必填	知识条目的完整正文
enabled	BOOLEAN	必填	知识条目是否参与智能问答检索
updated_at	DATETIME(3)	必填	知识条目最后更新时间

建表代码

```
CREATE TABLE `knowledge_item` (  
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '知识条目主键',  
  `knowledge_code` VARCHAR(64) NOT NULL COMMENT '知识条目的唯一业务编码',  
  `category` VARCHAR(60) NOT NULL COMMENT '知识分类，例如历史、结构、演奏技法或乐理',  
  `title` VARCHAR(200) NOT NULL COMMENT '知识条目标题',  
  `content` MEDIUMTEXT NOT NULL COMMENT '知识条目的完整正文',  
  `enabled` BOOLEAN NOT NULL DEFAULT TRUE COMMENT '知识条目是否参与智能问答检索',  
  `updated_at` DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3) COMMENT '知识条目最后更新时间',  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `uk_knowledge_code` (`knowledge_code`),  
  KEY `idx_knowledge_category` (`category`),  
  FULLTEXT KEY `ft_knowledge_text` (`title`, `content`)  
) ENGINE=InnoDB COMMENT='古筝历史与乐理知识库';
```

3.17 qa_answer

保存大模型针对一次用户提问生成的回答。

字段名	数据类型	定义提示	字段含义
id	BIGINT	PK / 必填 / 自增	智能问答结果主键
utterance_id	BIGINT	FK / UK / 必填	该回答对应的用户提问编号
answer_text	MEDIUMTEXT	必填	大模型生成的回答正文
model_name	VARCHAR(100)	可选/按定义	生成回答所使用的大模型名称
answered_at	DATETIME(3)	必填	回答生成完成时间

建表代码

```
CREATE TABLE `qa_answer` (  
  `id` BIGINT UNSIGNED NOT NULL AUTO INCREMENT COMMENT '智能问答结果主键',  
  `utterance_id` BIGINT UNSIGNED NOT NULL COMMENT '该回答对应的用户提问编号',  
  `answer_text` MEDIUMTEXT NOT NULL COMMENT '大模型生成的回答正文',  
  `model_name` VARCHAR(100) DEFAULT NULL COMMENT '生成回答所使用的大模型名称',  
  `answered_at` DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) COMMENT '回答生成完成时间',  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `uk_answer_utterance` (`utterance_id`),  
  CONSTRAINT `fk_answer_utterance` FOREIGN KEY (`utterance_id`)  
    REFERENCES `utterance` (`id`) ON DELETE CASCADE  
) ENGINE=InnoDB COMMENT='智能问答结果';
```

3.18 answer_source

记录一次回答引用了哪些知识条目及其相关程度。

字段名	数据类型	定义提示	字段含义
answer_id	BIGINT	联合PK / FK / 联合UK / 必填	引用知识来源的回答编号
knowledge_item_id	BIGINT	联合PK / FK / 必填	被回答引用的知识条目编号
rank_no	SMALLINT	联合UK / 必填	知识条目在本次检索结果中的排名
relevance_score	DECIMAL(5,4)	可选/按定义	知识条目与问题的相关度，取值范围为 0 到 1

建表代码

```
CREATE TABLE `answer_source` (  
  `answer_id` BIGINT UNSIGNED NOT NULL COMMENT '引用知识来源的回答编号',  
  `knowledge_item_id` BIGINT UNSIGNED NOT NULL COMMENT '被回答引用的知识条目编号',  
  `rank_no` SMALLINT UNSIGNED NOT NULL COMMENT '知识条目在本次检索结果中的排名',  
  `relevance_score` DECIMAL(5,4) DEFAULT NULL COMMENT '知识条目与问题的相关度，取值范围为 0 到 1',  
  PRIMARY KEY (`answer_id`, `knowledge_item_id`),  
  UNIQUE KEY `uk_answer_source_rank` (`answer_id`, `rank_no`),  
  KEY `idx_answer_source_knowledge` (`knowledge_item_id`),  
  CONSTRAINT `fk_answer_source_answer` FOREIGN KEY (`answer_id`)  
    REFERENCES `qa_answer` (`id`) ON DELETE CASCADE,  
  CONSTRAINT `fk_answer_source_knowledge` FOREIGN KEY (`knowledge_item_id`)  
    REFERENCES `knowledge_item` (`id`) ON DELETE RESTRICT,  
  CONSTRAINT `ck_answer_source_score` CHECK (`relevance_score` IS NULL OR `relevance_score` BETWEEN 0.0000 AND 1.0000)  
) ENGINE=InnoDB COMMENT='问答所引用的知识条目';
```

3.19 discovery_request

统一记录曲库浏览、歌曲搜索、直接点歌和智能推荐请求。

字段名	数据类型	定义提示	字段含义
id	BIGINT	PK / 必填 / 自增	歌曲发现请求主键
session_id	BIGINT	FK	发起请求的体验会话编号
utterance_id	BIGINT	FK	触发请求的文字或语音输入编号，浏览曲库时可为空
request_kind	VARCHAR(20)	必填	请求类型：浏览、搜索、直接点歌、推荐或替代推荐
status	VARCHAR(16)	必填	请求处理状态：等待、完成、无匹配或失败
min_match_score	DECIMAL(5,4)	可选/按定义	模糊匹配最低分值，例如 0.3000 表示 30%
created_at	DATETIME(3)	必填	请求创建时间
completed_at	DATETIME(3)	可选/按定义	请求处理完成时间

建表代码

```
CREATE TABLE `discovery_request` (  
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '歌曲发现请求主键',  
  `session_id` BIGINT UNSIGNED DEFAULT NULL COMMENT '发起请求的体验会话编号',  
  `utterance_id` BIGINT UNSIGNED DEFAULT NULL COMMENT '触发请求的文字或语音输入编号，浏览曲库时可为空',  
  `request_kind` VARCHAR(20) NOT NULL COMMENT '请求类型：浏览、搜索、直接点歌、推荐或替代推荐',  
  `status` VARCHAR(16) NOT NULL DEFAULT 'PENDING' COMMENT '请求处理状态：等待、完成、无匹配或失败',  
  `min_match_score` DECIMAL(5,4) DEFAULT NULL COMMENT '模糊匹配最低分值，例如 0.3000 表示 30%',  
  `created_at` DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) COMMENT '请求创建时间',  
  `completed_at` DATETIME(3) DEFAULT NULL COMMENT '请求处理完成时间',  
  PRIMARY KEY (`id`),  
  KEY `idx_discovery_session_time` (`session_id`, `created_at`),  
  KEY `idx_discovery_utterance` (`utterance_id`),  
  CONSTRAINT `fk_discovery_session` FOREIGN KEY (`session_id`)  
    REFERENCES `experience_session` (`id`) ON DELETE SET NULL,  
  CONSTRAINT `fk_discovery_utterance` FOREIGN KEY (`utterance_id`)  
    REFERENCES `utterance` (`id`) ON DELETE RESTRICT,  
  CONSTRAINT `ck_discovery_kind` CHECK (`request_kind` IN ('BROWSE','SEARCH','DIRECT','RECOMMEND','ALTERNATIVE')),  
  CONSTRAINT `ck_discovery_status` CHECK (`status` IN ('PENDING','COMPLETED','NO_MATCH','FAILED')),  
  CONSTRAINT `ck_discovery_score` CHECK (`min_match_score` IS NULL OR `min_match_score` BETWEEN 0.0000 AND 1.0000),  
  CONSTRAINT `ck_discovery_time` CHECK (`completed_at` IS NULL OR `completed_at` >= `created_at`),  
  CONSTRAINT `ck_discovery_input` CHECK (`request_kind` = 'BROWSE' OR `utterance_id` IS NOT NULL)  
) ENGINE=InnoDB COMMENT='曲库浏览、搜索与推荐请求';
```


3.20 discovery_candidate

保存一次搜索或推荐返回的候选歌曲和排序结果。

字段名	数据类型	定义提示	字段含义
id	BIGINT	PK / 必填 / 自增	候选歌曲结果主键
request_id	BIGINT	FK / 联合UK / 必填	候选结果所属的歌曲发现请求编号
song_id	BIGINT	FK / 联合UK / 必填	候选歌曲编号
rank_no	SMALLINT	联合UK / 必填	歌曲在本次候选列表中的排名
match_score	DECIMAL(5,4)	可选/按定义	歌曲与用户需求的匹配分值，取值范围为 0 到 1
candidate_role	VARCHAR(16)	必填	候选用途：直接匹配、智能推荐或替代歌曲
selected_at	DATETIME(3)	可选/按定义	用户选择该候选歌曲的时间，未选择时为空

建表代码

```
CREATE TABLE `discovery_candidate` (  
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '候选歌曲结果主键',  
  `request_id` BIGINT UNSIGNED NOT NULL COMMENT '候选结果所属的歌曲发现请求编号',  
  `song_id` BIGINT UNSIGNED NOT NULL COMMENT '候选歌曲编号',  
  `rank_no` SMALLINT UNSIGNED NOT NULL COMMENT '歌曲在本次候选列表中的排名',  
  `match_score` DECIMAL(5,4) DEFAULT NULL COMMENT '歌曲与用户需求的匹配分值，取值范围为 0 到 1',  
  `candidate_role` VARCHAR(16) NOT NULL COMMENT '候选用途：直接匹配、智能推荐或替代歌曲',  
  `selected_at` DATETIME(3) DEFAULT NULL COMMENT '用户选择该候选歌曲的时间，未选择时为空',  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `uk_candidate_song` (`request_id`, `song_id`),  
  UNIQUE KEY `uk_candidate_rank` (`request_id`, `rank_no`),  
  KEY `idx_candidate_song` (`song_id`),  
  CONSTRAINT `fk_candidate_request` FOREIGN KEY (`request_id`)  
    REFERENCES `discovery_request` (`id`) ON DELETE CASCADE,  
  CONSTRAINT `fk_candidate_song` FOREIGN KEY (`song_id`)  
    REFERENCES `song` (`work_id`) ON DELETE RESTRICT,  
  CONSTRAINT `ck_candidate_score` CHECK (`match_score` IS NULL OR `match_score` BETWEEN 0.0000 AND 1.0000),  
  CONSTRAINT `ck_candidate_role` CHECK (`candidate_role` IN ('MATCH','RECOMMENDED','SUBSTITUTE'))  
) ENGINE=InnoDB COMMENT='一次搜索或推荐产生的歌曲候选';
```

3.21 composition

保存自由创作作品独有的编辑状态和节拍配置。

字段名	数据类型	定义提示	字段含义
work_id	BIGINT	PK / FK / 必填	自由创作对应的可演奏作品编号，同时也是本表主键
session_id	BIGINT	FK	创建该作品的体验会话编号
edit_status	VARCHAR(16)	必填	创作编辑状态：编辑中、已锁定或已编译
ticks_per_beat	SMALLINT	必填	每一拍包含的时间刻度数，用于精确记录音符位置
locked_at	DATETIME(3)	可选/按定义	用户确认并锁定乐谱的时间

建表代码

```
CREATE TABLE `composition` (  
  `work_id` BIGINT UNSIGNED NOT NULL COMMENT '自由创作对应的可演奏作品编号，同时也是本表主键',  
  `session_id` BIGINT UNSIGNED DEFAULT NULL COMMENT '创建该作品的体验会话编号',  
  `edit_status` VARCHAR(16) NOT NULL DEFAULT 'EDITING' COMMENT '创作编辑状态：编辑中、已锁定或已编译',  
  `ticks_per_beat` SMALLINT UNSIGNED NOT NULL DEFAULT 480 COMMENT '每一拍包含的时间刻度数，用于精确记录音符位置',  
  `locked_at` DATETIME(3) DEFAULT NULL COMMENT '用户确认并锁定乐谱的时间',  
  PRIMARY KEY (`work_id`),  
  KEY `idx_composition_session` (`session_id`),  
  CONSTRAINT `fk_composition_work` FOREIGN KEY (`work_id`)  
    REFERENCES `playable_work` (`id`) ON DELETE CASCADE,  
  CONSTRAINT `fk_composition_session` FOREIGN KEY (`session_id`)  
    REFERENCES `experience_session` (`id`) ON DELETE SET NULL,  
  CONSTRAINT `ck_composition_status` CHECK (`edit_status` IN ('EDITING','LOCKED','COMPILED')),  
  CONSTRAINT `ck_composition_ticks` CHECK (`ticks_per_beat` > 0)  
) ENGINE=InnoDB COMMENT='自由创作专有信息';
```

3.22 ai_completion

记录自由创作中的一次 AI 智能补全批次。

字段名	数据类型	定义提示	字段含义
composition_id	BIGINT	联合PK / FK / 必填	使用智能补全的自由创作作品编号
batch_no	INT	联合PK / 必填	同一作品内的 AI 补全批次编号
anchor_tick	INT	必填	AI 从乐谱哪个时间刻度开始续写
status	VARCHAR(16)	必填	补全结果状态：已生成、已接受或已拒绝
model_name	VARCHAR(100)	可选/按定义	执行智能补全的模型名称
created_at	DATETIME(3)	必填	补全结果生成时间
decided_at	DATETIME(3)	可选/按定义	用户接受或拒绝补全结果的时间

建表代码

```
CREATE TABLE `ai_completion` (  
  `composition_id` BIGINT UNSIGNED NOT NULL COMMENT '使用智能补全的自由创作作品编号',  
  `batch_no` INT UNSIGNED NOT NULL COMMENT '同一作品内的 AI 补全批次编号',  
  `anchor_tick` INT UNSIGNED NOT NULL COMMENT 'AI 从乐谱哪个时间刻度开始续写',  
  `status` VARCHAR(16) NOT NULL DEFAULT 'GENERATED' COMMENT '补全结果状态：已生成、已接受或已拒绝',  
  `model_name` VARCHAR(100) DEFAULT NULL COMMENT '执行智能补全的模型名称',  
  `created_at` DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) COMMENT '补全结果生成时间',  
  `decided_at` DATETIME(3) DEFAULT NULL COMMENT '用户接受或拒绝补全结果的时间',  
  PRIMARY KEY (`composition_id`, `batch_no`),  
  KEY `idx_ai_completion_work_time` (`composition_id`, `created_at`),  
  CONSTRAINT `fk_ai_completion_work` FOREIGN KEY (`composition_id`)  
  REFERENCES `composition` (`work_id`) ON DELETE CASCADE,  
  CONSTRAINT `ck_ai_completion_status` CHECK (`status` IN ('GENERATED','ACCEPTED','REJECTED')),  
  CONSTRAINT `ck_ai_completion_time` CHECK (`decided_at` IS NULL OR `decided_at` >= `created_at`)  
) ENGINE=InnoDB COMMENT='一次智能补全批次';
```

3.23 composition_note

保存人工拖拽或 AI 生成的每一个音符事件。

字段名	数据类型	定义提示	字段含义
id	BIGINT	PK / 必填 / 自增	音符事件主键
composition_id	BIGINT	FK / 联合UK / 必填	音符所属的自由创作作品编号
string_part_id	BIGINT	FK / 联合UK / 必填	该音符需要机器人弹奏的琴弦编号
ai_completion_no	INT	联合FK	生成该音符的 AI 补全批次，人工音符为空
sequence_no	INT	联合UK / 必填	音符在编辑列表中的顺序编号
start_tick	INT	联合UK / 必填	音符开始演奏的时间刻度
duration_tick	INT	必填	音符持续的时间刻度数量
velocity	TINYINT	必填	演奏力度，取值范围为 1 到 127
note_state	VARCHAR(12)	必填	音符状态：建议中、已采用或已移除

建表代码

```
CREATE TABLE `composition_note` (  
  `id` BIGINT UNSIGNED NOT NULL AUTO INCREMENT COMMENT '音符事件主键',  
  `composition_id` BIGINT UNSIGNED NOT NULL COMMENT '音符所属的自由创作作品编号',  
  `string_part_id` BIGINT UNSIGNED NOT NULL COMMENT '该音符需要机器人弹奏的琴弦编号',  
  `ai_completion_no` INT UNSIGNED DEFAULT NULL COMMENT '生成该音符的 AI 补全批次, 人工音符为空',  
  `sequence_no` INT UNSIGNED NOT NULL COMMENT '音符在编辑列表中的顺序编号',  
  `start_tick` INT UNSIGNED NOT NULL COMMENT '音符开始演奏的时间刻度',  
  `duration_tick` INT UNSIGNED NOT NULL COMMENT '音符持续的时间刻度数量',  
  `velocity` TINYINT UNSIGNED NOT NULL DEFAULT 80 COMMENT '演奏力度, 取值范围为 1 到 127',  
  `note_state` VARCHAR(12) NOT NULL DEFAULT 'ACTIVE' COMMENT '音符状态: 建议中、已采用或已移除',  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `uk_composition_sequence` (`composition_id`, `sequence_no`),  
  UNIQUE KEY `uk_composition_tick_string` (`composition_id`, `start_tick`, `string_part_id`),  
  KEY `idx_note_string` (`string_part_id`),  
  CONSTRAINT `fk_note_composition` FOREIGN KEY (`composition_id`)  
  REFERENCES `composition` (`work_id`) ON DELETE CASCADE,  
  CONSTRAINT `fk_note_string` FOREIGN KEY (`string_part_id`)  
  REFERENCES `string_profile` (`part_id`) ON DELETE RESTRICT,  
  CONSTRAINT `fk_note_ai_completion` FOREIGN KEY (`composition_id`, `ai_completion_no`)  
  REFERENCES `ai_completion` (`composition_id`, `batch_no`) ON DELETE RESTRICT,  
  CONSTRAINT `ck_note_duration` CHECK (`duration_tick` > 0),  
  CONSTRAINT `ck_note_velocity` CHECK (`velocity` BETWEEN 1 AND 127),  
  CONSTRAINT `ck_note_state` CHECK (`note_state` IN ('PROPOSED','ACTIVE','REMOVED')),  
  CONSTRAINT `ck_note_origin` CHECK (  
    (`ai_completion_no` IS NULL AND `note_state` IN ('ACTIVE','REMOVED'))  
    OR (`ai_completion_no` IS NOT NULL)  
  )  
  ) ENGINE=InnoDB COMMENT='自由创作音符事件';
```

3.24 performance_run

统一记录来自探秘、点歌、语音或自由创作的机器人演奏任务。

字段名	数据类型	定义提示	字段含义
id	BIGINT	PK / 必填 / 自增	机器人演奏任务主键
session_id	BIGINT	FK	发起演奏的体验会话编号
work_id	BIGINT	FK / 必填	需要机器人演奏的统一作品编号
origin_module	VARCHAR(20)	必填	演奏来源模块：探秘、点歌、语音或自由创作
run_status	VARCHAR(16)	必填	演奏状态：排队、发送、演奏中、成功、失败或取消
requested_at	DATETIME(3)	必填	用户请求演奏的时间
started_at	DATETIME(3)	可选/按定义	机器人实际开始演奏的时间
ended_at	DATETIME(3)	可选/按定义	机器人结束演奏的时间

建表代码

```
CREATE TABLE `performance_run` (  
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '机器人演奏任务主键',  
  `session_id` BIGINT UNSIGNED DEFAULT NULL COMMENT '发起演奏的体验会话编号',  
  `work_id` BIGINT UNSIGNED NOT NULL COMMENT '需要机器人演奏的统一作品编号',  
  `origin_module` VARCHAR(20) NOT NULL COMMENT '演奏来源模块：探秘、点歌、语音或自由创作',  
  `run_status` VARCHAR(16) NOT NULL DEFAULT 'QUEUED' COMMENT '演奏状态：排队、发送、演奏中、成功、失败或取消',  
  `requested_at` DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) COMMENT '用户请求演奏的时间',  
  `started_at` DATETIME(3) DEFAULT NULL COMMENT '机器人实际开始演奏的时间',  
  `ended_at` DATETIME(3) DEFAULT NULL COMMENT '机器人结束演奏的时间',  
  PRIMARY KEY (`id`),  
  KEY `idx_run_session_time` (`session_id`, `requested_at`),  
  KEY `idx_run_work_time` (`work_id`, `requested_at`),  
  KEY `idx_run_status_time` (`run_status`, `requested_at`),  
  CONSTRAINT `fk_run_session` FOREIGN KEY (`session_id`)  
    REFERENCES `experience_session` (`id`) ON DELETE SET NULL,  
  CONSTRAINT `fk_run_work` FOREIGN KEY (`work_id`)  
    REFERENCES `playable_work` (`id`) ON DELETE RESTRICT,  
  CONSTRAINT `ck_run_origin` CHECK (`origin_module` IN ('EXPLORE','SONGBOOK','VOICE','COMPOSITION')),  
  CONSTRAINT `ck_run_status` CHECK (`run_status` IN ('QUEUED','SENDING','PLAYING','SUCCEEDED','FAILED','CANCELLED')),  
  CONSTRAINT `ck_run_time_order` CHECK (  
    (`started_at` IS NULL OR `started_at` >= `requested_at`)  
    AND (`ended_at` IS NULL OR (`started_at` IS NOT NULL AND `ended_at` >= `started_at`))  
  )  
) ENGINE=InnoDB COMMENT='统一机器人演奏任务';
```

3.25 robot_dispatch

记录机器人控制指令的发送、确认与失败重试。

字段名	数据类型	定义提示	字段含义
id	BIGINT	PK / 必填 / 自增	机器人指令发送记录主键
performance_run_id	BIGINT	FK / 联合UK / 必填	指令对应的机器人演奏任务编号
attempt_no	SMALLINT	联合UK / 必填	当前演奏任务的第几次发送尝试
command_asset_id	BIGINT	FK	编译后机器人控制指令的资源编号
dispatch_status	VARCHAR(16)	必填	发送状态：等待、已发送、已确认或失败
error_message	VARCHAR(1000)	可选/按定义	指令发送失败时记录的错误信息
sent_at	DATETIME(3)	可选/按定义	控制指令发送时间
acknowledged_at	DATETIME(3)	可选/按定义	机器人返回接收确认的时间

建表代码

```
CREATE TABLE `robot_dispatch` (  
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '机器人指令发送记录主键',  
  `performance_run_id` BIGINT UNSIGNED NOT NULL COMMENT '指令对应的机器人演奏任务编号',  
  `attempt_no` SMALLINT UNSIGNED NOT NULL COMMENT '当前演奏任务的第几次发送尝试',  
  `command_asset_id` BIGINT UNSIGNED DEFAULT NULL COMMENT '编译后机器人控制指令的资源编号',  
  `dispatch_status` VARCHAR(16) NOT NULL DEFAULT 'PENDING' COMMENT '发送状态：等待、已发送、已确认或失败',  
  `error_message` VARCHAR(1000) DEFAULT NULL COMMENT '指令发送失败时记录的错误信息',  
  `sent_at` DATETIME(3) DEFAULT NULL COMMENT '控制指令发送时间',  
  `acknowledged_at` DATETIME(3) DEFAULT NULL COMMENT '机器人返回接收确认的时间',  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `uk_dispatch_attempt` (`performance_run_id`, `attempt_no`),  
  KEY `idx_dispatch_asset` (`command_asset_id`),  
  KEY `idx_dispatch_status` (`dispatch_status`, `sent_at`),  
  CONSTRAINT `fk_dispatch_run` FOREIGN KEY (`performance_run_id`)  
    REFERENCES `performance_run` (`id`) ON DELETE CASCADE,  
  CONSTRAINT `fk_dispatch_asset` FOREIGN KEY (`command_asset_id`)  
    REFERENCES `digital_asset` (`id`) ON DELETE SET NULL,  
  CONSTRAINT `ck_dispatch_attempt` CHECK (`attempt_no` > 0),  
  CONSTRAINT `ck_dispatch_status` CHECK (`dispatch_status` IN ('PENDING','SENT','ACKED','FAILED')),  
  CONSTRAINT `ck_dispatch_time` CHECK (`acknowledged_at` IS NULL OR (`sent_at` IS NOT NULL AND `acknowledged_at` >= `sent_at`))  
) ENGINE=InnoDB COMMENT='机器人指令发送与重试记录';
```

3.26 performance_feedback

保存用户对一次机器人演奏的评分和文字评价。

字段名	数据类型	定义提示	字段含义
id	BIGINT	PK / 必填 / 自增	演奏反馈主键
performance_run_id	BIGINT	FK / UK / 必填	反馈对应的机器人演奏任务编号
rating	TINYINT	可选/按定义	用户评分，取值范围为 1 到 5
comment_text	VARCHAR(1000)	可选/按定义	用户填写的文字评价
created_at	DATETIME(3)	必填	反馈提交时间

建表代码

```
CREATE TABLE `performance_feedback` (  
  `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT COMMENT '演奏反馈主键',  
  `performance_run_id` BIGINT UNSIGNED NOT NULL COMMENT '反馈对应的机器人演奏任务编号',  
  `rating` TINYINT UNSIGNED DEFAULT NULL COMMENT '用户评分，取值范围为 1 到 5',  
  `comment_text` VARCHAR(1000) DEFAULT NULL COMMENT '用户填写的文字评价',  
  `created_at` DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3) COMMENT '反馈提交时间',  
  PRIMARY KEY (`id`),  
  UNIQUE KEY `uk_feedback_run` (`performance_run_id`),  
  CONSTRAINT `fk_feedback_run` FOREIGN KEY (`performance_run_id`)  
    REFERENCES `performance_run` (`id`) ON DELETE CASCADE,  
  CONSTRAINT `ck_feedback_rating` CHECK (`rating` IS NULL OR `rating` BETWEEN 1 AND 5)  
) ENGINE=InnoDB COMMENT='一次演奏后的总体反馈';
```


3.27 feedback_descriptor

保存用户为一次演奏选择的感受标签。

字段名	数据类型	定义提示	字段含义
feedback_id	BIGINT	联合PK / FK / 必填	演奏反馈编号
descriptor_id	BIGINT	联合PK / FK / 必填	用户为本次演奏选择的感受词编号

建表代码

```
CREATE TABLE `feedback_descriptor` (  
  `feedback_id` BIGINT UNSIGNED NOT NULL COMMENT '演奏反馈编号',  
  `descriptor_id` BIGINT UNSIGNED NOT NULL COMMENT '用户为本次演奏选择的感受词编号',  
  PRIMARY KEY (`feedback_id`, `descriptor_id`),  
  KEY `idx_feedback_descriptor_word` (`descriptor_id`),  
  CONSTRAINT `fk_feedback_descriptor_feedback` FOREIGN KEY (`feedback_id`)  
    REFERENCES `performance_feedback` (`id`) ON DELETE CASCADE,  
  CONSTRAINT `fk_feedback_descriptor_word` FOREIGN KEY (`descriptor_id`)  
    REFERENCES `descriptor` (`id`) ON DELETE RESTRICT  
) ENGINE=InnoDB COMMENT='用户选择的演奏感受词';
```